22. ROS2 Gazebo simulation platform

1. Introduction to Gazebo

Gazebo is a 3D physics simulation platform that is commonly used in robot development. It can simulate robots, sensors, environments and dynamic characteristics. This tutorial uses Gazebo Harmonic (Gazebo Sim 8), which goes with Ubuntu 24.04 and ROS2 Jazzy, and integrates with ROS2 through `ros_gz`.

Gazebo Classic has reached the end of its maintenance. The `gazebo_ros`, `spawn_entity.py`, `libgazebo_ros_init.so` and `libgazebo_ros_factory.so` used in the Humble tutorial belong to Gazebo Classic, and cannot be used any further in the Jazzy environment of this chapter.

2. Installation and environment check

First load the ROS2 Jazzy environment:

```
source /opt/ros/jazzy/setup.bash
```

Install the integration package of ROS2 and Gazebo:

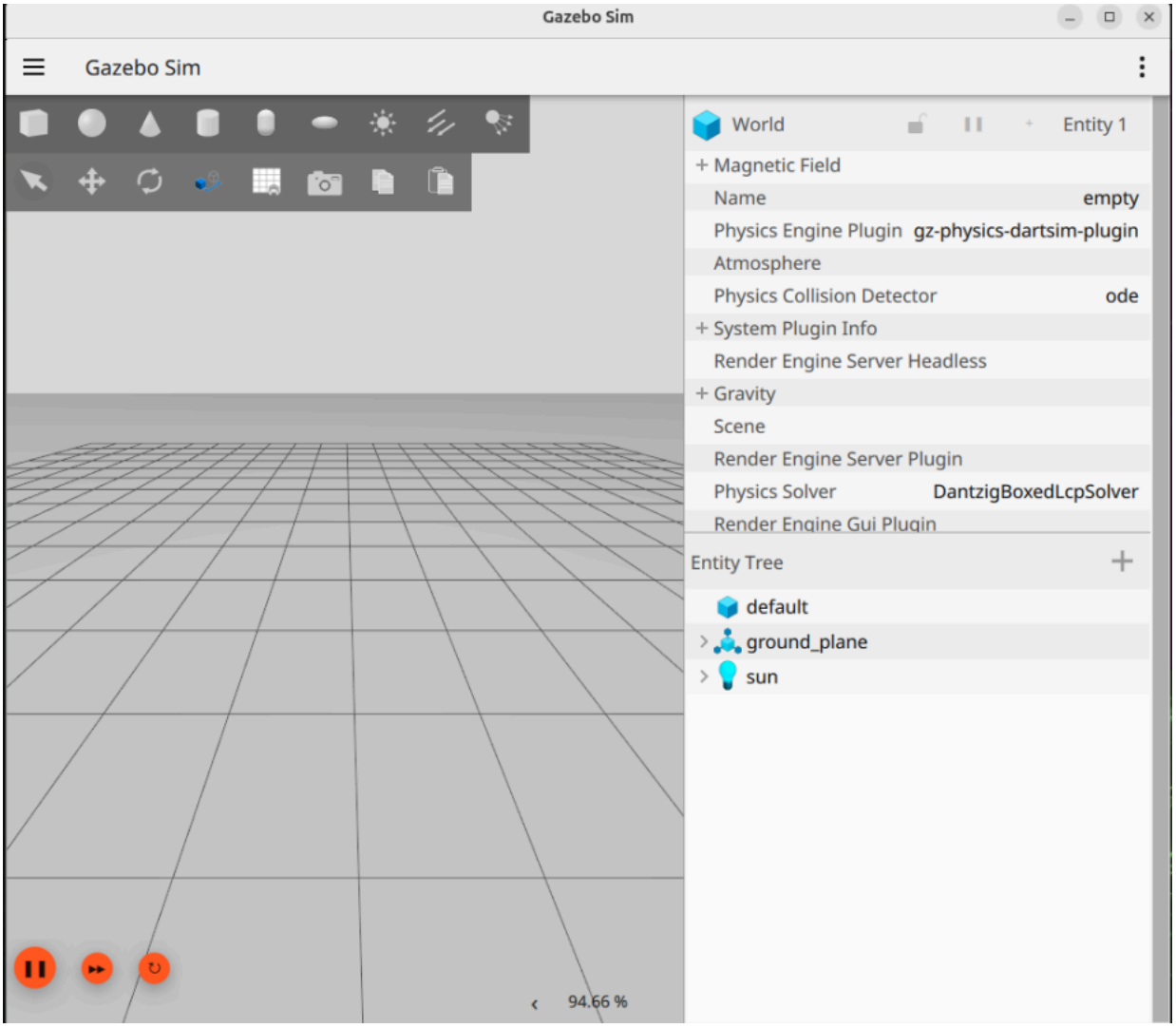
```
sudo apt update
sudo apt install ros-${ROS_DISTRO}-ros-gz
```

Check the installation result:

```
gz sim --versions
ros2 pkg prefix ros_gz_sim
```

The version adapted by this tutorial is Gazebo Sim 8 / Gazebo Harmonic. Start the empty world:

```
gz sim -r empty.sdf
```



`empty.sdf` is the empty world resource that comes with the installed Gazebo. Do not run `gz sim -s -r world.sdf` directly unless `world.sdf` really exists in the current directory or in the Gazebo resource path, otherwise you get `Unable to find or download file`.

It is recommended to run the Gazebo GUI in the VNC graphical desktop. When an SSH or RDP terminal does not have a correct `DISPLAY`, you can use the server-only mode to check:

```
gz sim -s -r empty.sdf
```

On a device with little memory, do not start several Gazebo instances at the same time; close the other simulation worlds such as TurtleBot3 before testing this chapter.

3. The relationship between Gazebo and ROS2

Gazebo Harmonic uses Gazebo Transport internally. When you only run `gz sim`, the Gazebo nodes do not necessarily appear in `ros2node list`, which is normal. When you have to pass topics between ROS2 and Gazebo, you should start `ros_gz_bridge` according to the message type; this chapter uses `ros_gz_sim create` to load the model into Gazebo directly.

You can check the Gazebo entity creation service with the following command:

```
gz service -l | grep create
```

4. Creating the function package

Enter the source directory of the ROS2 workspace and create the `myrobot` function package:

```
source /opt/ros/jazzy/setup.bash
mkdir -p ~/workspace/src
cd ~/workspace/src
ros2 pkg create myrobot --build-type ament_cmake \
  --dependencies launch launch_ros ros_gz_sim
cd myrobot
mkdir -p launch urdf
```

Create `demo01_base.urdf` in the `urdf` directory. This example sets the red material directly in the `visual` of the URDF and no longer uses the `Gazebo/Red` material script of Gazebo Classic:

```
<?xml version="1.0"?>
<robot name="myrobot">
  <link name="base_link">
    <visual>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
        <box size="0.2 0.2 0.2"/>
      </geometry>
      <material name="red">
        <color rgba="1 0 0 1"/>
      </material>
    </visual>
    <collision>
      <origin xyz="0 0 0" rpy="0 0 0"/>
      <geometry>
        <box size="0.2 0.2 0.2"/>
      </geometry>
    </collision>
    <inertial>
      <mass value="0.1"/>
      <inertia ixx="0.0006667" ixy="0" ixz="0"
               iyy="0.0006667" iyz="0" izz="0.0006667"/>
    </inertial>
  </link>
</robot>
```

5. Writing the Launch startup file

Create `bringup_model.launch.py` in the `launch` directory. This startup file first calls the Gazebo startup file provided by `ros_gz_sim`, and then loads the URDF model with the `create` program:

```
import os

from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, IncludeLaunchDescription
```

```
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node


def generate_launch_description():
    package_share = get_package_share_directory("myrobot")
    urdf_model_path = os.path.join(
        package_share, "urdf", "demo01_base.urdf"
    )

    gz_args = DeclareLaunchArgument(
        "gz_args",
        default_value="-r -v 3 empty.sdf",
        description="The startup arguments passed to Gazebo Sim",
    )

    start_gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(
                get_package_share_directory("ros_gz_sim"),
                "launch",
                "gz_sim.launch.py",
            )
        ),
        launch_arguments={"gz_args": LaunchConfiguration("gz_args")}.items(),
    )

    spawn_model = Node(
        package="ros_gz_sim",
        executable="create",
        arguments=[
            "-name", "myrobot",
            "-file", urdf_model_path,
            "-x", "0",
            "-y", "0",
            "-z", "0.2",
        ],
        output="screen",
    )

    return LaunchDescription([
        gz_args,
        start_gazebo,
        spawn_model,
    ])
```

6. Configuring the install rules

Add the following before `ament_package()` in `CMakeLists.txt`:

```
install(
  DIRECTORY launch urdf
  DESTINATION share/${PROJECT_NAME}
)
```

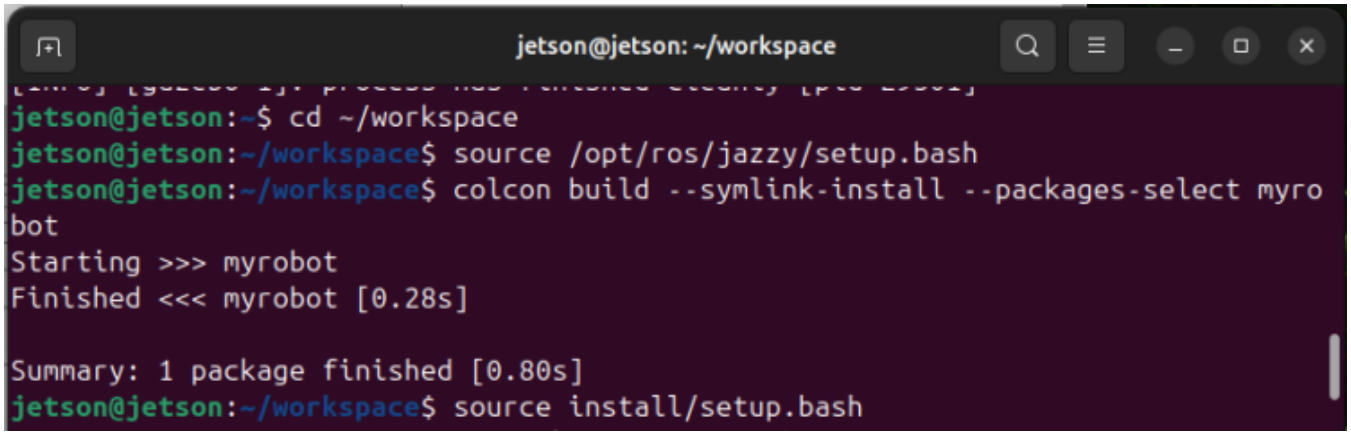
Confirm that `package.xml` contains the following runtime dependencies; they are written automatically when the package is created with the `ros2 pkg create --dependencies` command above:

```
<exec_depend>launch</exec_depend>
<exec_depend>launch_ros</exec_depend>
<exec_depend>ros_gz_sim</exec_depend>
```

7. Building and verifying the run

Go back to the root of the workspace and compile:

```
cd ~/workspace
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install --packages-select myrobot
source install/setup.bash
```



First check whether the function package and the Launch arguments have been installed:

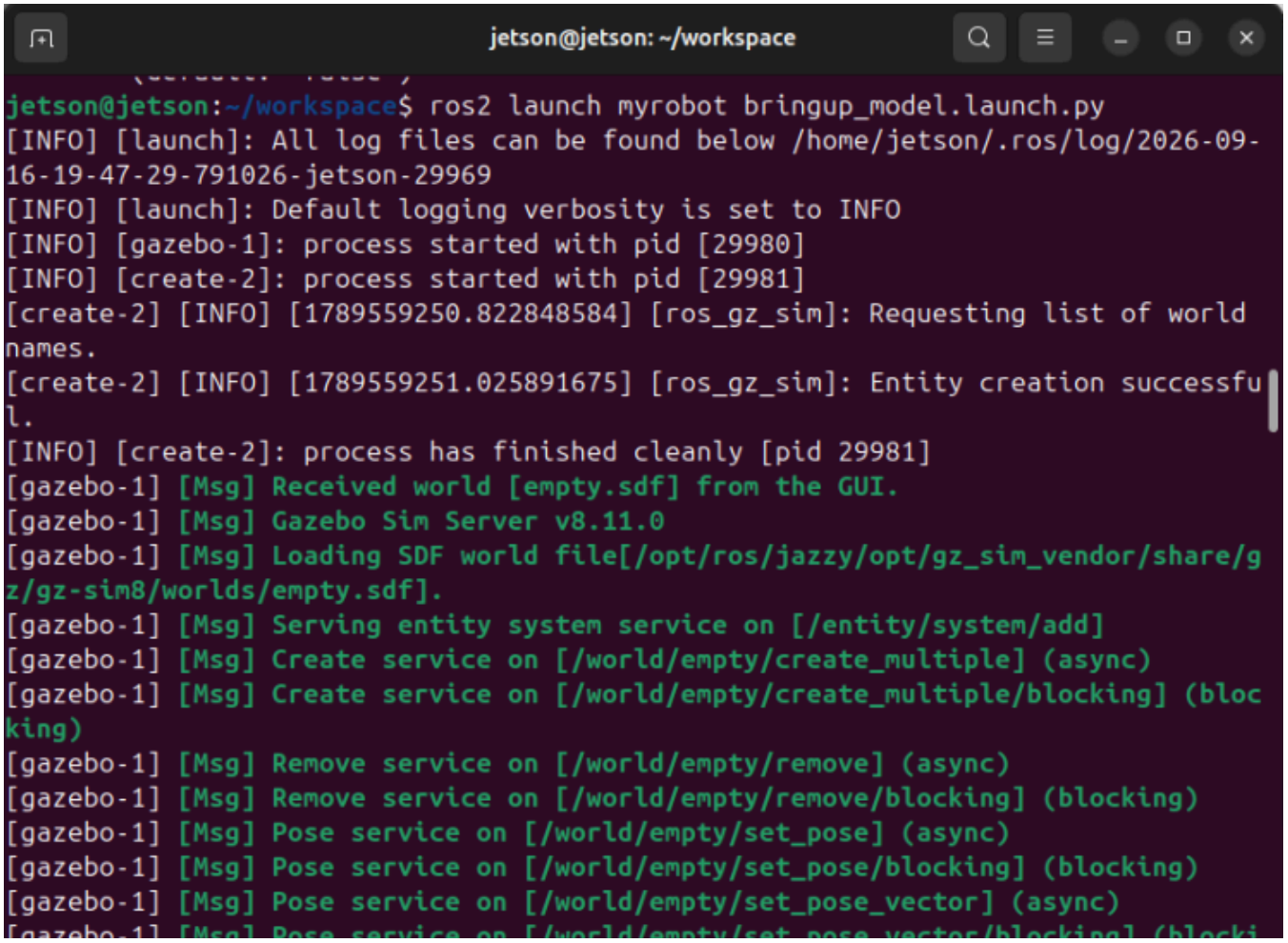
```
ros2 pkg prefix myrobot
ros2 launch myrobot bringup_model.launch.py --show-args
```

Start it in the VNC graphical desktop:

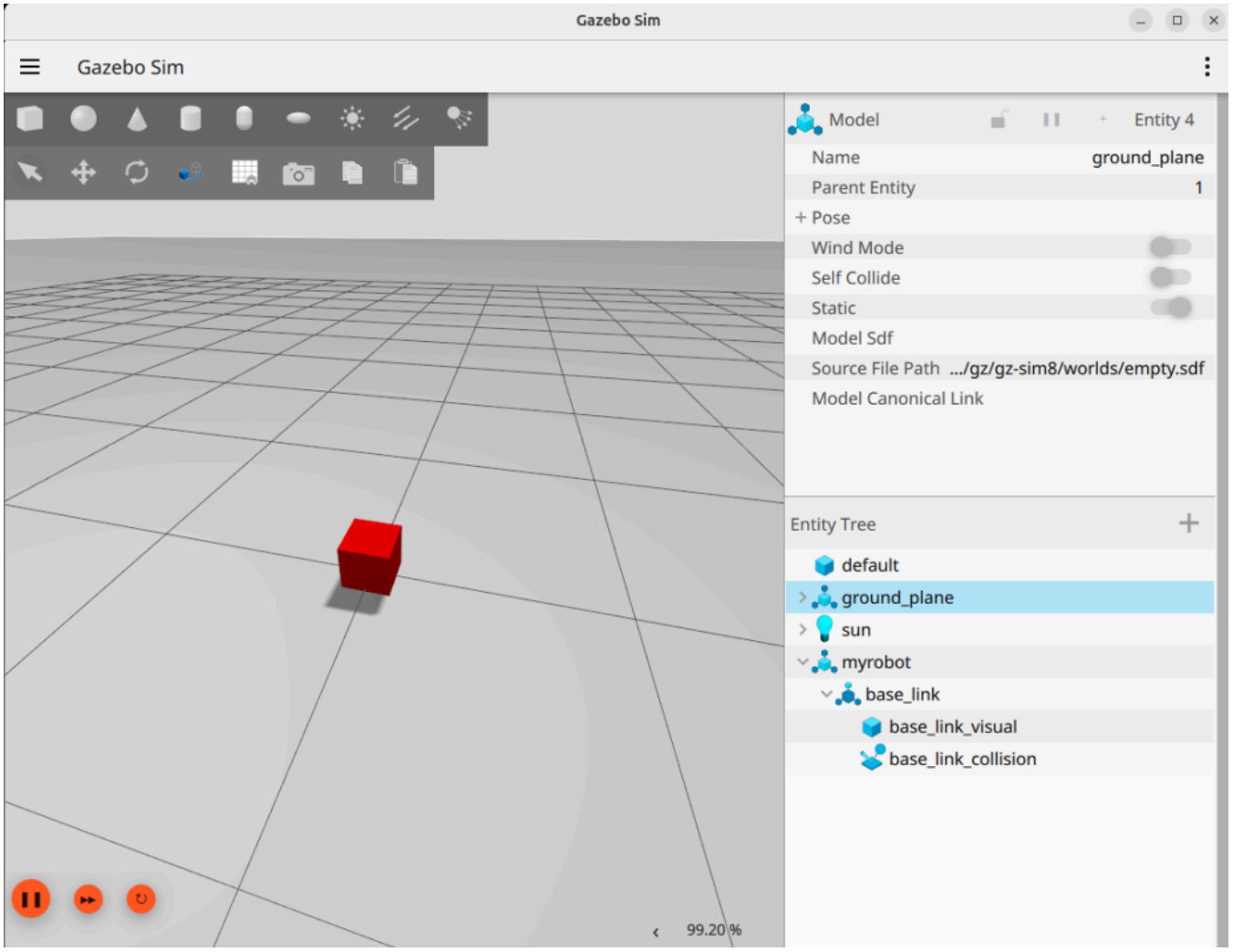
```
ros2 launch myrobot bringup_model.launch.py
```

Under normal circumstances the terminal prints:

```
[ros_gz_sim]: Entity creation successful.
```



A red cube appears in the Gazebo empty world. If the interface shows the TurtleBot3 world or green obstacles, other Gazebo processes are still running; close the other simulations first and then start only the Launch file of this chapter.



When there is no graphical desktop, you can verify it in server-only mode:

```
ros2 launch myrobot bringup_model.launch.py \
  gz_args:='-s -r -v 2 empty.sdf'
```

Seeing `Entity creation successful.` means that the Gazebo server has been started and the model has been loaded successfully; end the program with `Ctrl+C`.